

Lezione 1: Esercitazione Pratica

Collezioni, Interfacce Funzionali e Generics in Java

Michele Gentile

5 Maggio 2026

1 Inverti lista e massimo

🔗 Ripasso dal PDF originale

Un classico esercizio sui Generics per familiarizzare con la sintassi degli argomenti di tipo limitati.

Implementare i seguenti due metodi generici statici:

- **inverti**: data in input una lista di elementi di tipo generico, restituisca un'altra lista con gli elementi in ordine invertito.
- **max**: data in input una lista di elementi di tipo generico, ne restituisca il valore massimo.

💡 Tip: Sintassi dei Bounded Types

Per il metodo **max**, dovete usare il costrutto `<T extends Comparable<T>`. Questo impone che il tipo passato sia effettivamente confrontabile con se stesso.

2 Sistema di Monitoraggio Sensori

1. Si definisca una classe **Sensore** i cui oggetti sono costruiti con un **codice seriale** (String), una **tipologia** (es. "Temperatura") e una **zona** di installazione.
2. Si crei una classe **ReteSensori** che contiene un insieme di sensori utilizzando inizialmente un **HashSet**.
3. Implementare correttamente **equals** e **hashCode** nella classe *Sensore* affinché due sensori siano considerati uguali se e solo se hanno lo stesso codice seriale.
4. Si implementi una versione chiamata **ReteOrdinata** che utilizzi un **TreeSet** per mantenere i sensori ordinati per **zona** e, a parità di zona, per **tipologia**.

🔗 Obiettivo: OOP, HashSet e TreeSet

L'obiettivo è gestire l'unicità di oggetti complessi basandosi su identificativi e definire criteri di ordinamento multipli.

💡 Tip: Uguaglianza vs Ordinamento

Ricordate: `HashSet` usa `equals/hashCode`, mentre `TreeSet` si basa su `compareTo` (interfaccia `Comparable`). Assicuratevi che siano coerenti tra loro per evitare bug logici!

3 Pipeline di Elaborazione Prezzi

Creare una classe **CalcolatoreFiscale** costruita con una lista di funzioni (`Function<Double, Double>`) da applicare in sequenza a un valore decimale.

- La classe deve avere un metodo `applicaTasse(Double valore)` che restituisce il risultato finale dopo tutta la pipeline.
- Testate la classe creando un'istanza con le seguenti lambda (o riferimenti a metodo):
 1. Sconto del 10% ($\times 0.9$).
 2. Arrotondamento all'intero superiore (`Math.ceil`).
 3. Aggiunta quota fissa di 5.50.
 4. Calcolo IVA al 22% ($\times 1.22$).

🔗 Obiettivo: Interfacce Funzionali e Lambda

Sperimentiamo l'uso di `java.util.function.Function` per creare sequenze di operazioni dinamiche.

4 Analisi Generica di Liste

Implementare i seguenti due metodi generici statici:

1. **filtraCompresi**: riceve una lista di elementi `T` e due valori `min` e `max`. Restituisce una nuova lista con gli elementi x tali che $min \leq x \leq max$.
2. **trovaElementoDominante**: riceve una lista di elementi `T` e restituisce l'elemento che compare più spesso (suggerimento: usate una `Map<T, Integer>`).

🔗 Obiettivo: Metodi Generici Bounded

Scrittura di utility riutilizzabili per diversi tipi di dati confrontabili.

⚠️ Attenzione: Vincoli sui tipi

Per il metodo `filtraCompresi` è necessario che il tipo `T` implementi l'interfaccia `Comparable`, altrimenti non potrete usare il metodo `compareTo`.